

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph dan Tree

Oleh:

Achmad Sajidi Aziz NIM. 2510817210025

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph dan Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Achmad Sajidi Aziz
NIM : 2510817210025

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI	ii
DAFTAR GAMBAR.....	iii
DAFTAR TABEL	iv
Soal 1: Konversi Adjacency List ke Adjacency Matrix	1
A. Source Code.....	2
B. Output Program	4
C. Pembahasan	4
Soal 2: Konversi Adjacency Matrix ke Adjacency List	6
A. Source Code.....	7
B. Output Program	9
C. Pembahasan	9
Soal 3: BFS: Jarak Terpendek Keluar Labirin	11
A. Source Code.....	12
B. Output Program	15
C. Pembahasan	15
Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin	17
A. Source Code.....	18
B. Output Program	21
C. Pembahasan	21
Soal 5:	23
A. Source Code.....	24
B. Output Program	27
C. Pembahasan	27
TAUTAN GIT	29

DAFTAR GAMBAR

Gambar 1 Screenshot Hasil Jawaban Soal 1	4
Gambar 2 Screenshot Hasil Jawaban Soal 2	9
Gambar 3 Screenshot Hasil Jawaban Soal 3	15
Gambar 4 Screenshot Hasil Jawaban Soal 4	21
Gambar 5 Screenshot Hasil Jawaban Soal 5	27

DAFTAR TABEL

Tabel 1 Source Code prob1.cpp.....	2
Tabel 2 Source Code prob2.cpp.....	7
Tabel 3 Source Code prob3.cpp.....	12
Tabel 4 Source Code prob4.cpp.....	18
Tabel 5 Source Code prob5.cpp.....	24

Soal 1: Konversi Adjacency List ke Adjacency Matrix

Deskripsi Soal

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
M
U1 V1 W1
U2 V2 W2
...
UM VM WM
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

```
V1 V2 ... VN
V1 a11 a12 ... a1N
V2 a21 a22 ... a2N
...
VN aN1 aN2 ... aNN
```

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

A. Source Code

Tabel 1 Source Code probl.cpp

```
1 #include <iostream>
2 #include <vector>
3 #include <unordered_map>
4
5 using namespace std;
6
7 int main() {
8     int N;
9     if (!(cin >> N)) return 0;
10
11     vector<char> vertices(N);
12     unordered_map<char, int> labelToIndex;
13
14     for (int i = 0; i < N; ++i) {
15         cin >> vertices[i];
16         labelToIndex[vertices[i]] = i;
17     }
18
19     int M;
20     cin >> M;
21
22     vector<vector<int>> matrix(N, vector<int>(N, 0));
23
24     for (int i = 0; i < M; ++i) {
25         char u, v;
26         int w;
27         cin >> u >> v >> w;
28
29         int u_idx = labelToIndex[u];
30         int v_idx = labelToIndex[v];
```

```

31
32     matrix[u_idx][v_idx] = w;
33 }
34
35 cout << "Adjacency Matrix:\n";
36
37 for (int i = 0; i < N; ++i) {
38     cout << vertices[i] << (i == N - 1 ? "" : " ");
39 }
40 cout << "\n";
41
42 for (int i = 0; i < N; ++i) {
43     cout << vertices[i];
44     for (int j = 0; j < N; ++j) {
45         cout << " " << matrix[i][j];
46     }
47     cout << "\n";
48 }
49
50 return 0;
51 }

```


B. Output Program

```
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
```

Gambar 1 Screenshot Hasil Jawaban Soal 1

C. Pembahasan

Soal pertama ini pada dasarnya meminta konversi representasi data pada sebuah graph berarah dan berbobot, dari bentuk awal adjacency list menjadi sebuah adjacency matrix. Algoritma yang digunakan lebih ke arah manipulasi struktur data dan pemetaan elemen dibandingkan algoritma pencarian rute yang rumit.

Teknik utamanya memanfaatkan kombinasi hash map dan array dua dimensi. Karena label vertex di sini menggunakan karakter unik seperti huruf kapital A sampai Z, karakter ini tidak bisa langsung digunakan sebagai indeks numerik untuk array matriks. Oleh karena itu, map seperti `unordered_map<char, int>` `labelToIndex` di dalam kode bertugas menerjemahkan huruf-huruf tersebut menjadi angka indeks yang berurutan agar mudah diposisikan ke dalam baris dan kolom tabel.

Alur kerjanya lumayan sederhana, dimulai dengan menyimpan semua label vertex dari input ke dalam array `vector<char>` `vertices` untuk menjaga urutan, lalu menghubungkannya ke masing-masing indeks di dalam map. Setelah itu, matriks dua dimensi `vector<vector<int>>` `matrix` disiapkan dan diisi penuh dengan angka 0 sebagai nilai standar yang menandakan belum ada edge. Terus, setiap kali program membaca data rute berupa titik asal, tujuan, dan bobot, program akan mencari letak indeksnya, lalu nilai 0 di koordinat matriks tersebut tinggal ditimpa langsung dengan angka bobotnya lewat perintah `matrix[u_idx][v_idx] = w`.

Syarat dari soal sebenarnya sangat menguntungkan karena graph ini sudah dijamin tidak memiliki self-loop dan bebas dari edge yang terduplikasi. Jadinya tidak butuh logika tambahan di dalam kode untuk mengecek apakah titik awal dan tujuannya sama, atau pusing

menentukan bobot mana yang harus diambil kalau datanya ganda. Sifat graph yang directed juga membuat pengisian matriksnya cukup dilakukan searah saja tanpa perlu repot mengisi indeks kebalikannya.

Jika dianalisis, kompleksitas waktunya akan berjalan di tingkat $O(N^2 + M)$. Hal ini karena program harus memproses data input jalan sebanyak M kali, lalu dilanjutkan dengan perulangan bersarang sebesar $N \times N$ untuk mencetak hasil akhir matriksnya secara utuh. Secara memori, alokasi matriks dua dimensi ini memakan ruang $O(N^2)$, yang sebenarnya lumayan boros memori jika graph tersebut memiliki sangat sedikit jalan penghubung, tetapi memang seperti itulah sifat asli dari pembentukan adjacency matrix.

Berhubung batas vertex N maksimal hanya 10 dan edge M mentok di angka 90, ukuran datanya benar-benar kecil. Melakukan perulangan untuk mencetak matriks 10×10 dipastikan akan selesai diproses dalam waktu sepersekian milidetik saja.

Soal 2: Konversi Adjacency Matrix ke Adjacency List

Deskripsi Soal

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
A11 A12 ... A1N
A21 A22 ... A2N
...
AN1 AN2 ... ANN
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

```
V1 -> (X1,w1), (X2,w2), ...
V2 -> (X1,w1), (X2,w2), ...
...
VN -> ...
```

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

A. Source Code

Tabel 2 Source Code prob2.cpp

```
1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 int main() {
7     int N;
8     if (!(cin >> N)) return 0;
9
10    vector<char> vertices(N);
11    for (int i = 0; i < N; ++i) {
12        cin >> vertices[i];
13    }
14
15    vector<vector<int>> matrix(N, vector<int>(N));
16    for (int i = 0; i < N; ++i) {
17        for (int j = 0; j < N; ++j) {
18            cin >> matrix[i][j];
19        }
20    }
21
22    cout << "Adjacency List:\n";
23    for (int i = 0; i < N; ++i) {
24        cout << vertices[i] << " ->";
25
26        bool has_edge = false;
27
28        for (int j = 0; j < N; ++j) {
29            if (matrix[i][j] > 0) {
30                if (has_edge) {
```

```
31         cout << ",";
32     }
33     cout << " (" << vertices[j] << "," <<
matrix[i][j] << ")";
34     has_edge = true;
35 }
36 }
37
38 if (!has_edge) {
39     cout << " -";
40 }
41
42 cout << "\n";
43 }
44
45 return 0;
46 }
```

B. Output Program

```
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
```

Gambar 2 Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Soal kedua ini mengubah representasi graph dari bentuk adjacency matrix kembali menjadi adjacency list melalui penelusuran struktur data dua dimensi secara linear. Algoritmanya tidak melibatkan pencarian rute yang rumit, melainkan murni menggunakan perulangan bersarang atau nested loop untuk memeriksa setiap sel matriks, lalu mencocokkan nilai numerik tersebut dengan label vertex asli yang ditampung berurutan dalam array vector char bernama vertices agar susunan output selaras dengan urutan input.

Alur eksekusinya berawal dari pembacaan deretan nama vertex dan penyusunan tabel vector dua dimensi bernama matrix berukuran $N \times N$. Penelusuran kemudian dilakukan baris demi baris menggunakan perulangan, dengan logika utama bertumpu pada blok kondisi `if (matrix[i][j] > 0)` untuk memicu pencetakan nama tujuan beserta bobot rutanya. Sebuah variabel penanda bool `has_edge` disisipkan di awal tiap baris khusus untuk mendeteksi vertex yang buntu, sehingga program bisa mengeksekusi pencetakan tanda strip jika kondisi `if (!has_edge)` terbukti benar di akhir baris.

Aturan soal yang menetapkan bahwa angka nol berarti tidak ada edge dan nilai diagonal dipastikan selalu nol sangat memangkas kebutuhan validasi ekstra. Hal ini otomatis mengeliminasi kewajiban untuk menambahkan logika khusus yang menangani rute berputar ke titik asalnya sendiri, karena nilai nol tersebut pasti akan langsung diabaikan oleh pengecekan nilai positif pada kondisi utama tadi.

Kebutuhan waktu pemrosesannya mutlak berjalan di tingkat $O(N^2)$ akibat keharusan menginspeksi seluruh elemen matriks melalui dua lapis perulangan for. Kebutuhan memori program juga sama-sama berada di tingkat $O(N^2)$ karena seluruh data nilai bobot dari input harus dialokasikan secara utuh ke dalam vector dua dimensi sebelum bisa dibongkar menjadi susunan list.

Mengingat batas maksimal jumlah vertex N hanya mencapai 10, dimensi matriks terbesarnya otomatis akan mentok di ukuran 10×10 elemen. Mengeksekusi maksimal 100 iterasi perulangan dan mengalokasikan 100 bilangan bulat ke dalam memori merupakan beban kerja komputasi yang teramat ringan, sehingga algoritma ini dipastikan berjalan dengan sangat instan dan efisien sesuai dengan skala masalah yang diberikan.

Soal 3: BFS: Jarak Terpendek Keluar Labirin

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS). Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22 ... G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

A. Source Code

Tabel 3 Source Code prob3.cpp

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4
5  using namespace std;
6
7  struct Point {
8      int r, c;
9  };
10
11 int main() {
12     ios_base::sync_with_stdio(false);
13     cin.tie(NULL);
14
15     int R, C;
16     if (!(cin >> R >> C)) return 0;
17
18     vector<vector<int>> grid(R, vector<int>(C));
19     for (int i = 0; i < R; ++i) {
20         for (int j = 0; j < C; ++j) {
21             cin >> grid[i][j];
22         }
23     }
24
25     int SR, SC, FR, FC;
26     cin >> SR >> SC >> FR >> FC;
27
28     vector<vector<int>> dist(R, vector<int>(C, -1));
29
30     queue<Point> q;
```

```

31
32     q.push({SR, SC});
33     dist[SR][SC] = 0;
34
35     int dr[] = {-1, 1, 0, 0};
36     int dc[] = {0, 0, -1, 1};
37
38     while (!q.empty()) {
39         Point curr = q.front();
40         q.pop();
41
42         if (curr.r == FR && curr.c == FC) {
43             break;
44         }
45
46         for (int i = 0; i < 4; ++i) {
47             int next_r = curr.r + dr[i];
48             int next_c = curr.c + dc[i];
49
50             if (next_r >= 0 && next_r < R && next_c >= 0
&& next_c < C) {
51                 if (grid[next_r][next_c] == 0 &&
dist[next_r][next_c] == -1) {
52                     dist[next_r][next_c] =
dist[curr.r][curr.c] + 1;
53                     q.push({next_r, next_c});
54                 }
55             }
56         }
57     }
58

```

```
59     cout << dist[FR][FC] << "\n";  
60  
61     return 0;  
62 }
```

B. Output Program

```
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 0 1
1 0 0 0 0 0 0 0 1 0
1 1 0 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 0 1
1 1 0 0 0 0 1 0 0 0
0 0
7 9
18
```

Gambar 3 Screenshot Hasil Jawaban Soal 3

C. Pembahasan

Soal ketiga ini berfokus pada pencarian jarak langkah minimum di dalam labirin berukuran $R \times C$ menggunakan algoritma Breadth-First Search atau BFS. Algoritma ini sangat ideal karena sifatnya yang mengeksplorasi rute secara berlapis atau level by level, dengan implementasi utama mengandalkan struktur data antrian berupa queue Point q untuk menampung koordinat sel yang sedang aktif diproses.

Alur eksekusi dimulai dengan memasukkan koordinat posisi awal ke dalam antrian dan mengunci nilai jarak awalnya menjadi 0 pada matriks vector dua dimensi bernama dist. Selama antrian belum kosong, program mengambil koordinat terdepan lewat fungsi q.front() lalu mengeksplorasi empat kotak tetangganya. Jika sebuah kotak berada di dalam area grid, bernilai jalan kosong atau 0, dan status dist-nya masih -1 yang berarti belum dikunjungi, kotak tersebut akan dimasukkan ke antrian dan jaraknya diperbarui lewat operasi $\text{dist}[\text{next_r}][\text{next_c}] = \text{dist}[\text{curr.r}][\text{curr.c}] + 1$.

Aturan soal yang melarang pergerakan diagonal diterapkan dengan praktis menggunakan array bantuan dr dan dc yang murni hanya memuat kombinasi perpindahan vertikal dan horizontal. Penggunaan angka -1 sebagai nilai inisialisasi awal pada seluruh sel matriks dist juga secara otomatis menuntaskan syarat penanganan jalan buntu, karena jika koordinat tujuan gagal dicapai sampai antrian habis, luaran program tinggal mencetak isi dari dist[FR][FC] yang nilainya dijamin tetap -1.

Setiap sel jalan bernilai 0 pada grid labirin dipastikan hanya akan dimasukkan ke dalam queue dan diproses pemeriksaannya maksimal tepat satu kali sepanjang perulangan berlangsung. Di sisi lain, eksekusi program perlu mengalokasikan array dua dimensi grid untuk memetakan labirin, matriks dist untuk mencatat rekam jejak jarak, serta kapasitas memori penampung antrian yang membengkak seiring bertambah luasnya area eksplorasi. Oleh karena itu, dapat disimpulkan bahwa algoritma ini memiliki kompleksitas waktu $O(R \times C)$ dan kompleksitas ruang $O(R \times C)$.

Mengingat batas ukuran maksimal baris R dan kolom C masing-masing hanya mentok di angka 100, total luas grid labirin terbesarnya tidak akan pernah melampaui 10.000 kotak. Menjalankan operasional antrean BFS dan mengalokasikan beberapa matriks penampung pada area grid berskala sekecil ini merupakan tugas komputasi yang teramat ringan, sehingga algoritma ini dipastikan selesai dieksekusi dengan sangat instan dan efisien sesuai dengan skala masalah yang diberikan.

Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22 ... G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

A. Source Code

Tabel 4 Source Code prob4.cpp

```
1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  int R, C;
7  int FR, FC;
8  int total_paths = 0;
9
10 int dr[] = {-1, 1, 0, 0};
11 int dc[] = {0, 0, -1, 1};
12
13 void dfs(int r, int c, const vector<vector<int>>& grid,
14         vector<vector<bool>>& visited) {
15     if (r == FR && c == FC) {
16         total_paths++;
17         return;
18     }
19     visited[r][c] = true;
20
21     for (int i = 0; i < 4; ++i) {
22         int next_r = r + dr[i];
23         int next_c = c + dc[i];
24
25         if (next_r >= 0 && next_r < R && next_c >= 0 &&
26             next_c < C) {
27             if (grid[next_r][next_c] == 0 &&
28                 !visited[next_r][next_c]) {
29                 dfs(next_r, next_c, grid, visited);
30             }
31         }
32     }
33 }
```

```

28         }
29     }
30 }
31
32     visited[r][c] = false;
33 }
34
35 int main() {
36     ios_base::sync_with_stdio(false);
37     cin.tie(NULL);
38
39     if (!(cin >> R >> C)) return 0;
40
41     vector<vector<int>> grid(R, vector<int>(C));
42     for (int i = 0; i < R; ++i) {
43         for (int j = 0; j < C; ++j) {
44             cin >> grid[i][j];
45         }
46     }
47
48     int SR, SC;
49     cin >> SR >> SC >> FR >> FC;
50
51     vector<vector<bool>> visited(R, vector<bool>(C,
false));
52
53     dfs(SR, SC, grid, visited);
54
55     cout << total_paths << "\n";
56
57     return 0;

```


58	}
----	---

B. Output Program

```
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
```

Gambar 4 Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Soal keempat ini beralih dari mencari rute tercepat menjadi menghitung total semua jalur sederhana dari posisi awal ke akhir pada grid labirin. Algoritma spesifik yang dipakai adalah Depth-First Search atau DFS rekursif yang dikombinasikan dengan teknik backtracking. Pendekatan ini sangat cocok karena DFS menggali sebuah rute sedalam mungkin pada graph sampai menemukan tujuan atau jalan buntu, lalu berjalan mundur untuk mencoba percabangan lain yang belum dieksplorasi.

Alur eksekusi bermula dari pemanggilan fungsi `dfs()` dengan koordinat awal, lalu sel tersebut langsung ditandai dengan `visited[r][c] = true` agar tidak dilewati dua kali. Program mencoba bergerak ke empat arah, dan jika target valid berupa jalan bernilai 0 serta berstatus `!visited`, fungsi memanggil dirinya sendiri secara rekursif ke sel baru. Saat kondisi `r == FR && c == FC` terpenuhi, variabel `total_paths` ditambah satu, lalu baris krusial `visited[r][c] = false` dieksekusi sebagai bentuk backtracking agar sel kembali bebas untuk rute alternatif.

Aturan soal yang melarang jalur mengunjungi sel yang sama lebih dari sekali langsung diselesaikan oleh matriks vector dua dimensi `visited` dan sistem backtracking. Penggunaan array `dr` dan `dc` yang murni memuat perpindahan vertikal dan horizontal juga memastikan tidak ada edge diagonal yang melanggar aturan. Syarat sel start dan finish yang dijamin bernilai 0 membebaskan program dari kewajiban melakukan validasi nilai ekstra di titik awal.

Dalam eksekusinya, program merangkai seluruh probabilitas jalur menggunakan pembentukan tree rekursi yang bercabang masif karena setiap vertex rata-rata memiliki hingga tiga opsi pergerakan valid. Di sisi lain, tumpukan pemanggilan rekursi terus memakan ruang memori yang berbanding lurus dengan kedalaman rute maksimal di dalam grid beserta ukuran matriks `visited`. Mengingat sifatnya yang mengeksplorasi seluruh kemungkinan cabang ini, dapat disimpulkan bahwa algoritma memiliki kompleksitas waktu $O(4^{(R \times C)})$ dan kompleksitas ruang $O(R \times C)$.

Meskipun memiliki sifat eksponensial, ukuran baris `R` dan kolom `C` sengaja dibatasi maksimal hanya di angka 7, membuat total grid hanya sebatas 49 kotak. Adanya jaminan

tambahan bahwa jumlah jalur valid tidak akan melebihi 100.000 membuat tree pencarian akan terputus wajar tanpa memicu beban pemrosesan berlebih. Menghitung puluhan ribu percabangan rekursi pada skala data sekecil ini merupakan tugas komputasi ringan, sehingga algoritma dipastikan selesai tereksekusi dengan sangat efisien dan instan

Soal 5: Tree Traversal

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah RedBlack Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

```
N
A1 A2 ... AN
Q
T1
T2
...
TQ
```

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

```
[Preorder] : daftar_angka
[Inorder] : daftar_angka
[Postorder] : daftar_angka
```

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:
Tree kosong. Tidak ada yang bisa ditampilkan.

A. Source Code

Tabel 5 Source Code prob5.cpp

```
1 #include <iostream>
2 #include <string>
3 #include "RedBlackTree.h"
4
5 using namespace std;
6
7 void preorder(const RedBlackTree::Node* node, const
RedBlackTree::Node* nilNode, bool& first) {
8     if (node == nilNode || node->isNil) return;
9     if (!first) cout << " ";
10    cout << node->key;
11    first = false;
12    preorder(node->left, nilNode, first);
13    preorder(node->right, nilNode, first);
14 }
15
16 void inorder(const RedBlackTree::Node* node, const
RedBlackTree::Node* nilNode, bool& first) {
17    if (node == nilNode || node->isNil) return;
18    inorder(node->left, nilNode, first);
19    if (!first) cout << " ";
20    cout << node->key;
21    first = false;
22    inorder(node->right, nilNode, first);
23 }
24
25 void postorder(const RedBlackTree::Node* node, const
RedBlackTree::Node* nilNode, bool& first) {
26    if (node == nilNode || node->isNil) return;
27    postorder(node->left, nilNode, first);
```

```

28     postorder(node->right, nilNode, first);
29     if (!first) cout << " ";
30     cout << node->key;
31     first = false;
32 }
33
34 int main() {
35     ios_base::sync_with_stdio(false);
36     cin.tie(NULL);
37
38     int N;
39     if (!(cin >> N)) return 0;
40
41     RedBlackTree tree;
42     for (int i = 0; i < N; ++i) {
43         int val;
44         cin >> val;
45         if (!tree.contains(val)) {
46             tree.insert(val);
47         }
48     }
49
50     int Q;
51     cin >> Q;
52     for (int i = 0; i < Q; ++i) {
53         string type;
54         cin >> type;
55
56         if (tree.empty()) {
57             cout << "Tree kosong. Tidak ada yang bisa
ditampilkan.\n";

```

```

58         continue;
59     }
60
61     if (type == "PREORDER" || type == "ALL") {
62         cout << "[Preorder] : ";
63         bool first = true;
64         preorder(tree.root(), tree.nil(), first);
65         cout << "\n";
66     }
67     if (type == "INORDER" || type == "ALL") {
68         cout << "[Inorder] : ";
69         bool first = true;
70         inorder(tree.root(), tree.nil(), first);
71         cout << "\n";
72     }
73     if (type == "POSTORDER" || type == "ALL") {
74         cout << "[Postorder]: ";
75         bool first = true;
76         postorder(tree.root(), tree.nil(), first);
77         cout << "\n";
78     }
79 }
80
81 return 0;
82 }

```

B. Output Program

```
7
50 30 70 20 40 60 80
4
PREORDER
INORDER
POSTORDER
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder]: 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder]: 20 40 30 60 80 70 50
```

Gambar 5 Screenshot Hasil Jawaban Soal 5

C. Pembahasan

Soal kelima ini berfokus pada operasi insert elemen dan traversal struktur data Red-Black Tree. Algoritmanya memanfaatkan sifat binary search tree untuk melakukan traversal rekursif dengan metode preorder, inorder, dan postorder. Teknik ini membaca urutan input angka untuk mem-build tree, lalu mencetak susunan node sesuai path traversal yang diminta oleh string query.

Alur eksekusi bermula dari perulangan untuk melakukan insert nilai ke dalam tree. Kondisi `if (!tree.contains(val))` dipakai khusus untuk memblokir angka duplikat agar perintah `tree.insert(val)` hanya mengeksekusi node yang bernilai unik. Setelah tree terbentuk, program memproses query dengan memanggil fungsi rekursif seperti preorder dan melempar parameter node root. Variabel boolean `first` disisipkan pada parameter fungsi untuk mencegah kelebihan spasi di akhir deretan angka.

Aturan pengabaian elemen duplikat langsung tuntas dieksekusi oleh pemanggilan fungsi `contains` sebelum proses insert node. Syarat pencetakan teks peringatan saat tree kosong juga difasilitasi dengan praktis lewat pengecekan kondisi `if (tree.empty())` di awal pemrosesan query. Selain itu, perintah query ALL dapat diakomodasi tanpa perlu merakit fungsi tambahan, cukup dengan memanggil ketiga fungsi traversal tadi secara sekuensial.

Operasi insert node ke dalam Red-Black Tree selalu melakukan self-balancing secara logaritmik sehingga memakan waktu pembentukan awal $O(N \log N)$. Saat menjawab query, fungsi rekursif wajib men-traverse seluruh node utuh satu kali, sehingga pemrosesan Q buah query memakan waktu eksekusi $O(Q \times N)$. Ruang memori juga wajib dialokasikan secara dinamis untuk menyimpan struktur node tree sekaligus menampung kedalaman call stack rekursi. Dari alur komputasi tersebut, dapat disimpulkan bahwa program ini memiliki kompleksitas waktu $O(N \log N + Q \times N)$ dan kompleksitas ruang $O(N)$.

Karena batas maksimal elemen N hanya 1000 dan jumlah query Q mentok di angka 20, beban komputasi traversal tertinggi hanya berkisar pada 20.000 operasi. Batasan input key yang nilainya bisa mencapai angka miliaran juga tetap aman ditampung oleh tipe integer standar tanpa ada risiko integer overflow. Mengalokasikan seribu node ke dalam heap memory dan men-traverse data tersebut berulang kali pada skala batasan ini merupakan tugas ringan, sehingga algoritma dipastikan tereksekusi secara instan dan amat efisien.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-asajidiaziz0-alt>